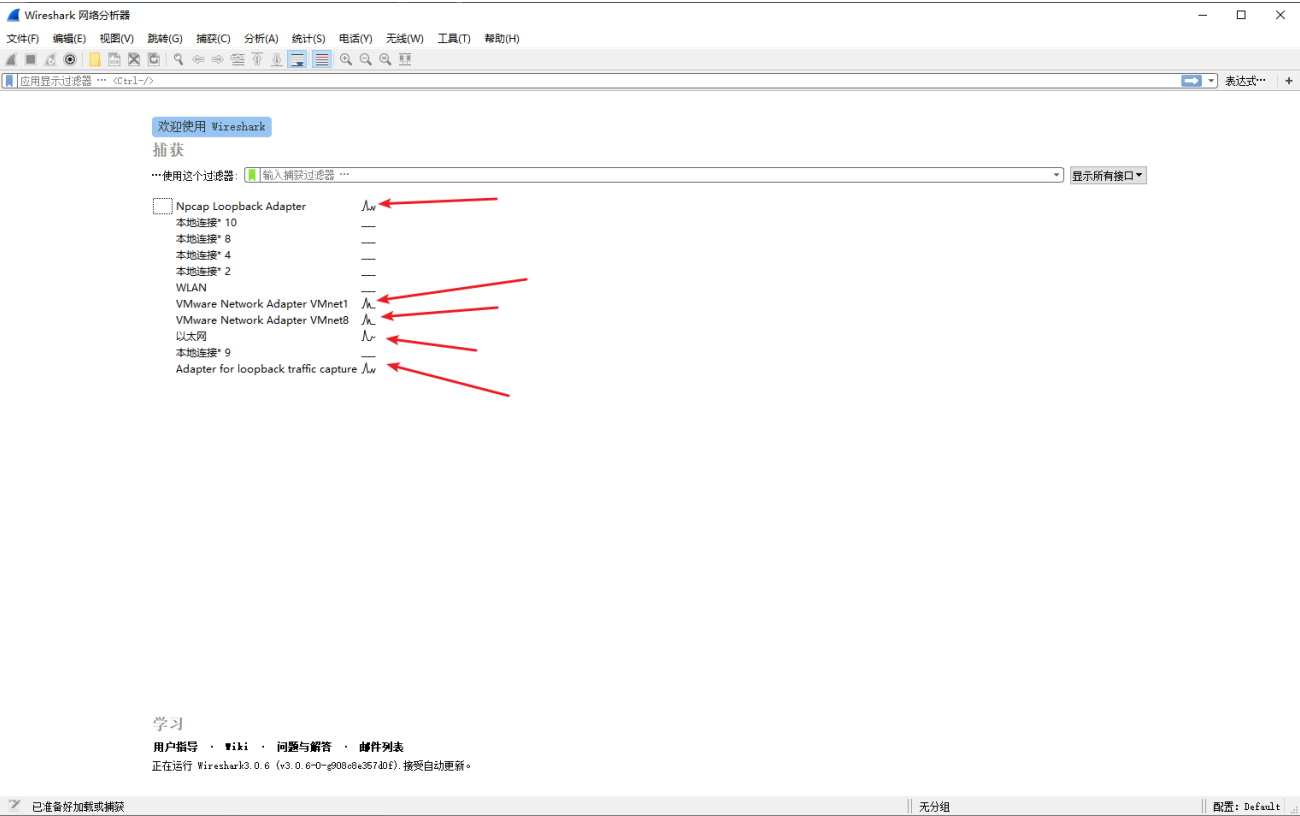
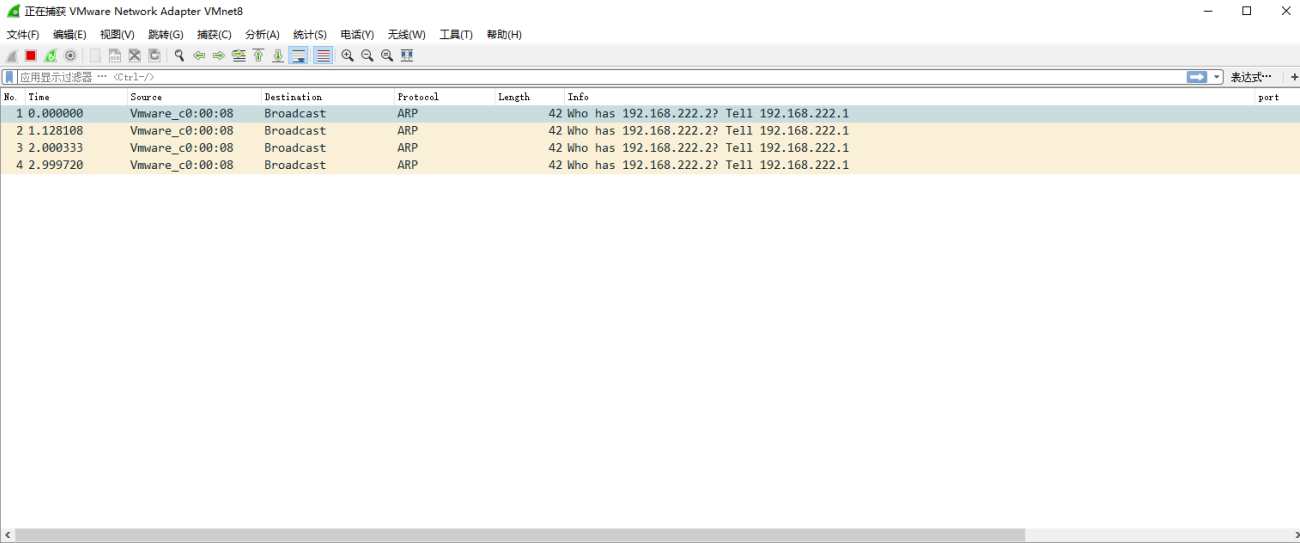


网络层各类协议数据的抓取分析

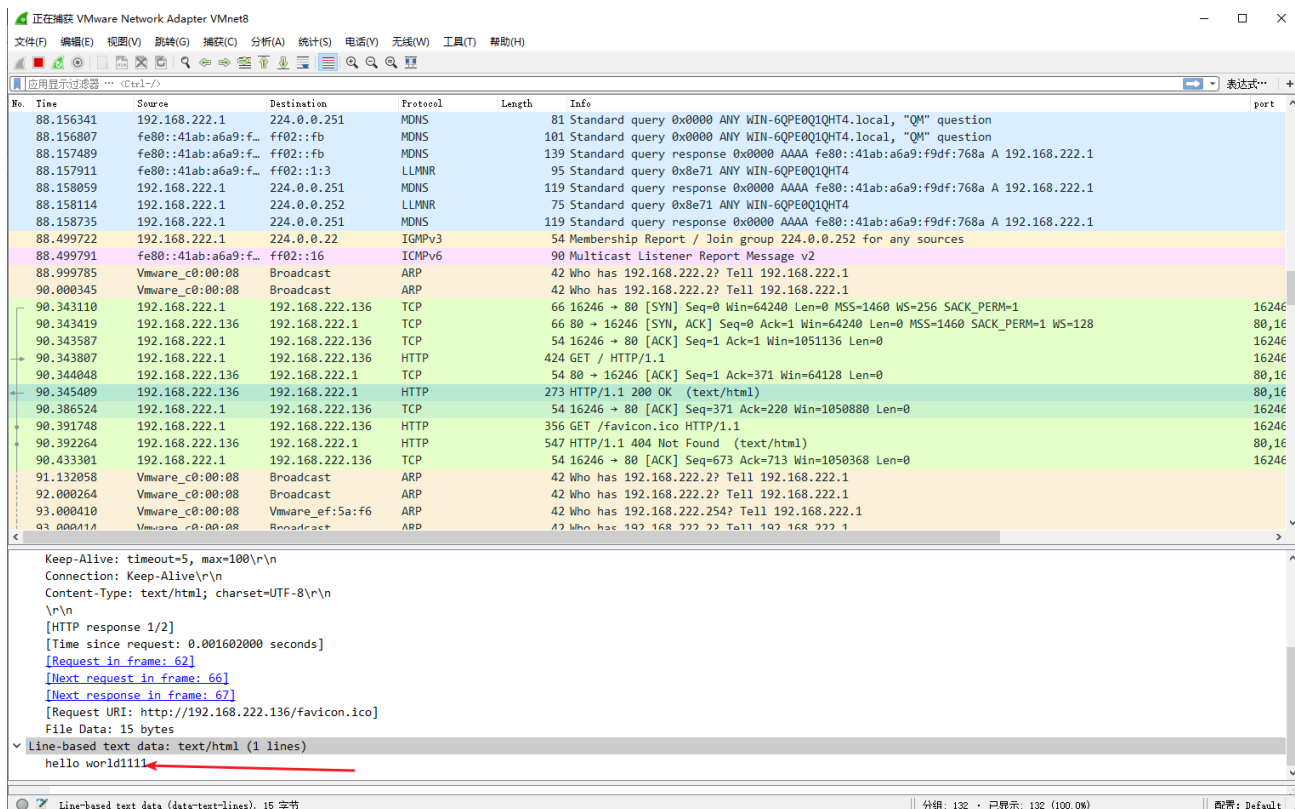
一、实验准备



打开 Wireshark，一些网卡有波动，就代表有抓到流量，选择一张有波动的网卡，进行抓包



可以看到已经抓到一些 arp 数据包了，可以通过访问网站，创造一些其他的流量



可以看到抓到了一些 http 和 tcp 的流量

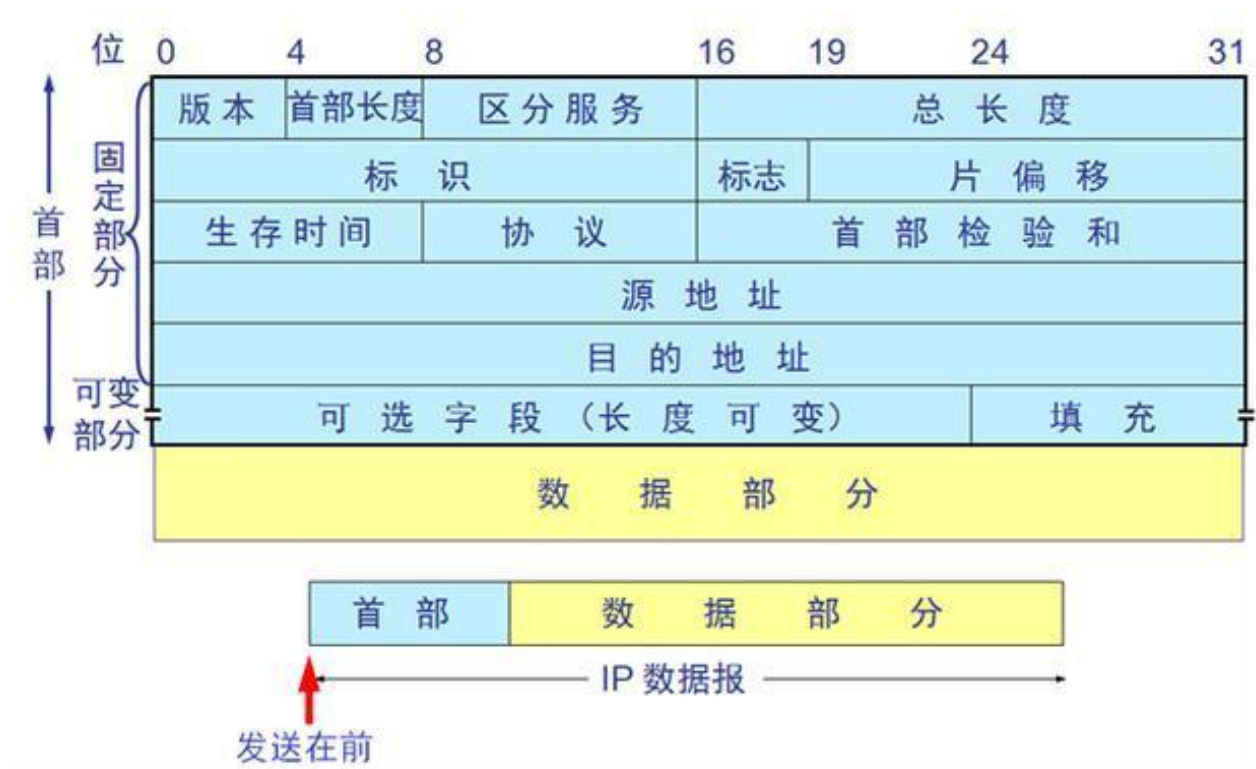
223.453209	192.168.222.1	192.168.222.136	ICMP	74 Echo (ping) request id=0x0001, seq=840/18435, ttl=128 (reply in 164)
223.453410	192.168.222.136	192.168.222.1	ICMP	74 Echo (ping) reply id=0x0001, seq=840/18435, ttl=64 (request in 163)
223.499743	Vmware_c0:00:08	Broadcast	ARP	42 Who has 192.168.222.2? Tell 192.168.222.1
224.456207	192.168.222.1	192.168.222.136	ICMP	74 Echo (ping) request id=0x0001, seq=841/18691, ttl=128 (reply in 167)
224.456780	192.168.222.136	192.168.222.1	ICMP	74 Echo (ping) reply id=0x0001, seq=841/18691, ttl=64 (request in 166)
224.637641	Vmware_c0:00:08	Broadcast	ARP	42 Who has 192.168.222.2? Tell 192.168.222.1
225.468497	192.168.222.1	192.168.222.136	ICMP	74 Echo (ping) request id=0x0001, seq=842/18947, ttl=128 (reply in 170)
225.469164	192.168.222.136	192.168.222.1	ICMP	74 Echo (ping) reply id=0x0001, seq=842/18947, ttl=64 (request in 169)
225.500212	Vmware_c0:00:08	Broadcast	ARP	42 Who has 192.168.222.2? Tell 192.168.222.1
226.474798	192.168.222.1	192.168.222.136	ICMP	74 Echo (ping) request id=0x0001, seq=843/19203, ttl=128 (reply in 173)
226.474967	192.168.222.136	192.168.222.1	ICMP	74 Echo (ping) reply id=0x0001, seq=843/19203, ttl=64 (request in 172)

ping 一下主机也可以抓到 icmp 协议

二、实验步骤

1. IP 协议分析

ip 协议分组格式



IP 协议首部包含两个部分，固定部分和可变部分，固定部分固定长 20byte

版本	占 4bit 表示版本信息 如 0100 表示 ipv4
首部长度	占 4bit 表示首部的总长度，0101 为 5，这里一个 1 表示 4byte, 所以头部占 20byte
服务类型	占 8bit 用来获得更好的服务。其中的前 3bit 表示报文的优先级，值越大优先级越高，最小 000，最大 111，后面的几位分别表示要求更低时延、更高的吞吐量、更高的可靠性、更低的路由代价等。对应位为 1 即有相应要求，为 0 则不要求。最后一位不启用，为 0
总长度	占 16bit 表示首部长度+数据长度，最大可以达到 65535byte 数据长度可以通过总长度-首部长度来计算
标识	占 16bit 唯一的标识主机发送的每一个数据报。其初始值由系统随机生成，每发送一个数据报其值就加一。该值在数据报分片的时候被复制到每个分片中，因此同一个数据报的所有分片都具有相同的标示值。
标志	占 3bit 三位标识符第一位保留，为 0，二三位为 DM 位，D 位如果为 1，表示不允许分片，M 位表示后面还有分片(除了最后一个分片，其他分片都要置 1)
片偏移	占 13bit 是分片相对原始 IP 数据报开始处的偏移，以 8 个 byte 为偏移单位
生存时间	占 8bit 即 TTL，为了限制数据报在网络中的生存时间，用跳数作为单位，每经过一个路由器就减 1，windwos 为 128，Linux 为 64 或 255
协议	占 8bit 指出此数据报携带的数据用什么协议处理
首部校验和	占 16bit 用来检验 IP 数据报头部在传输过程中是否损坏，只检测数据报的首部，不包括数据部分，不采用 crc 校验码，只采用简单的计算方法
源地址	占 32bit, 表示请求报文的地址
目标地址	占 32bit，表示报文的目的地地址

对前面抓到的一个 http 包进行分析

No.	Time	Source	Destination	Protocol	Length	Info	port
88.158114	192.168.222.1	224.0.0.252	LLMNR		75	Standard query 0x8e71 ANY WIN-6QPE0Q1QHT4	
88.158735	192.168.222.1	224.0.0.251	MDNS		119	Standard query response 0x0000 AAAA fe80::41ab:a6a9:f9df:768a A 192.168.222.1	
88.499722	192.168.222.1	224.0.0.22	IGMPv3		54	Membership Report / Join group 224.0.0.252 for any sources	
88.499791	fe80::41ab:a6a9:f9df:768a	ff02::16	ICMPv6		90	Multicast Listener Report Message v2	
88.999785	Vmware_c0:00:08	Broadcast	ARP		42	Who has 192.168.222.2? Tell 192.168.222.1	
90.000345	Vmware_c0:00:08	Broadcast	ARP		42	Who has 192.168.222.2? Tell 192.168.222.1	
90.343110	192.168.222.1	192.168.222.136	TCP		66	16246 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1	16246
90.343419	192.168.222.136	192.168.222.1	TCP		66	80 → 16246 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM=1 WS=128	80,16
90.343587	192.168.222.1	192.168.222.136	TCP		54	16246 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len=0	16246
90.343807	192.168.222.1	192.168.222.136	HTTP		424	GET / HTTP/1.1	16246
90.344048	192.168.222.136	192.168.222.1	TCP		54	80 → 16246 [ACK] Seq=1 Ack=371 Win=64128 Len=0	80,16
90.345409	192.168.222.136	192.168.222.1	HTTP		273	HTTP/1.1 200 OK (text/html)	80,16
90.386524	192.168.222.1	192.168.222.136	TCP		54	16246 → 80 [ACK] Seq=371 Ack=220 Win=1050880 Len=0	16246
90.391748	192.168.222.1	192.168.222.136	HTTP		356	GET /favicon.ico HTTP/1.1	16246
90.392264	192.168.222.136	192.168.222.1	HTTP		547	HTTP/1.1 404 Not Found (text/html)	80,16
90.433301	192.168.222.1	192.168.222.136	TCP		54	16246 → 80 [ACK] Seq=673 Ack=713 Win=1050368 Len=0	16246
91.132058	Vmware_c0:00:08	Broadcast	ARP		42	Who has 192.168.222.2? Tell 192.168.222.1	
92.000264	Vmware_c0:00:08	Broadcast	ARP		42	Who has 192.168.222.2? Tell 192.168.222.1	
93.000410	Vmware_c0:00:08	Vmware_ef:5a:f6	ARP		42	Who has 192.168.222.254? Tell 192.168.222.1	
93.000414	Vmware_c0:00:08	Broadcast	ARP		42	Who has 192.168.222.2? Tell 192.168.222.1	
93.000471	Vmware_ef:5a:f6	Vmware_c0:00:08	ARP		42	192.168.222.254 is at 00:50:56:ef:5a:f6	
94.132744	Vmware_c0:00:08	Broadcast	ARP		42	Who has 192.168.222.2? Tell 192.168.222.1	
95.000345	Vmware_c0:00:08	Broadcast	ARP		42	Who has 192.168.222.2? Tell 192.168.222.1	
95.398007	192.168.222.136	192.168.222.1	TCP		54	80 → 16246 [FIN, ACK] Seq=713 Ack=673 Win=64128 Len=0	80,16
95.398125	192.168.222.1	192.168.222.136	TCP		54	16246 → 80 [ACK] Seq=673 Ack=714 Win=1050368 Len=0	16246

Keep-Alive: timeout=5, max=100\r\n

Wireshark · 分组 64 · VMware Network Adapter VMnet8

```
> Frame 64: 273 bytes on wire (2184 bits), 273 bytes captured (2184 bits) on interface 0
> Ethernet II, Src: Vmware_fa:d5:99 (00:0c:29:fa:d5:99), Dst: Vmware_c0:00:08 (00:50:56:c0:00:08)
> Internet Protocol Version 4, Src: 192.168.222.136, Dst: 192.168.222.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
      .... 00.. = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 259
  Identification: 0xf724 (63268)
  > Flags: 0x4000, Don't fragment
    0... .. = Reserved bit: Not set
    .1.. .. = Don't fragment: Set
    ..0. .. = More fragments: Not set
    ...0 0000 0000 0000 = Fragment offset: 0
  Time to live: 64
  Protocol: TCP (6)
```

```
0000 00 50 56 c0 00 08 00 0c 29 fa d5 99 08 00 45 00 PV.....).....E-
0010 01 03 f7 24 40 00 40 06 04 f5 c0 a8 de 88 c0 a8 ...$@.....
0020 de 01 00 50 3f 76 46 1a 00 f9 1a c8 7e c0 50 18 ...P?vf-...P-
0030 01 f5 fc d3 00 00 48 54 54 50 2f 31 2e 31 20 32 .....HT TP/1.1 2
0040 30 30 20 4f 4b 0d 0a 44 61 74 65 3a 20 54 68 75 00 OK..D ate: Thu
0050 2c 20 33 31 20 4f 63 74 20 32 30 31 39 20 30 37 , 31 Oct 2019 07
0060 3a 32 31 3a 31 38 20 47 4d 54 0d 0a 53 65 72 76 :21:18 GMT- Serv
0070 65 72 3a 20 41 70 61 63 68 65 2f 32 2e 34 2e 32 er: Apache/2.4.2
0080 39 20 28 55 62 75 6e 74 75 29 0d 0a 43 6f 6e 74 9 (Ubuntu)-Cont
0090 65 6e 74 2d 4c 65 6e 67 74 68 3a 20 31 35 0d 0a ent-Leng th: 15-
00a0 4b 65 65 70 2d 41 6c 69 76 65 3a 20 74 69 6d 65 Keep-Ali ve: time
00b0 6f 75 74 3d 35 2c 20 6d 61 78 3d 31 30 30 0d 0a out=5, m ax=100-
00c0 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 4b 65 65 70 Connecti on: Keep
00d0 2d 41 6c 69 76 65 0d 0a 43 6f 6e 74 65 6e 74 2d -Alive- Content-
00e0 54 79 70 65 3a 20 74 65 78 74 2f 68 74 6d 6c 3b Type: te xt/html;
00f0 20 63 68 61 72 73 65 74 3d 55 54 46 2d 38 0d 0a charset =UTF-8-
0100 0d 0a 68 61 6c 6c 6f 20 77 6f 72 6c 64 31 31 31 ..hello world111
0110 31 1
```

Close

Help

打开可以看到里面的详细信息，对其 ip 协议进行分析

Internet Protocol Version 4, Src: 192.168.222.136, Dst: 192.168.222.1

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

▼ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

0000 00.. = Differentiated Services Codepoint: Default (0)

.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)

Total Length: 259

Identification: 0xf724 (63268)

▼ Flags: 0x4000, Don't fragment

0... .. = Reserved bit: Not set

.1.. .. = Don't fragment: Set

..0. = More fragments: Not set

...0 0000 0000 0000 = Fragment offset: 0

Time to live: 64

Protocol: TCP (6)

Header checksum: 0x04f5 [validation disabled]

[Header checksum status: Unverified]

Source: 192.168.222.136

Destination: 192.168.222.1

表示版本为ipv4

表示首部长度为20byte, 5表示5个单位, 一个单位为4byte

区分服务领域, 前6为以0结尾, 表示由Internet指定的一个区分服务类型, 后二位未启用

表示总长度为259个byte

表示标识, 如果有相同标识的, 就表示是同一个数据包

第一行是保留位, 默认为0, 第二行为1, 表示不允许分片, 第三行为0表示后面没有更多的包了。

片偏移为0表示只有一个包或者这是第一个包, 所以偏移值是0

ttl表示存活时间, 这里64表示没有经过路由器

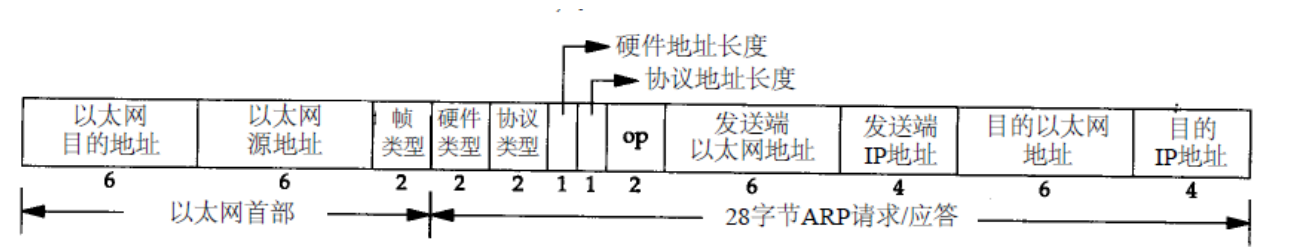
表示数据要交给tcp协议处理

这里的首部校验和关闭, 因为wireshark默认关闭这个选项, 正常情况下应该是good, bad.

表示源地址和目的地址

2. ARP 协议分析

ARP 协议分组格式



目的地址	占 6 个 byte, 为广播请求报文全为 1, 即 ff:ff:ff:ff:ff:ff, 响应报文为目标 mac 地址
源地址	占 6 个 byte, 为源 mac 地址
帧类型	占 2 个 byte, 表示后面数据的类型, 对于 ARP 请求或应答来说, 该字段的值为 0x0806
硬件类型	占 2 个 byte, 硬件类型, 为 1 是由以太网指派
协议类型	使用协议的类型 0x8000 是 tcp
硬件地址长度	6
协议长度	4
操作码	为 1 是请求包, 2 是响应包
发送方 mac 地址	发送方 mac 地址
发送方 ip 地址	发送方 ip 地址
目标 mac 地址	目标 mac 地址, 如果目标 mac 未知, 全用 0 标识
目标 ip 地址	目标 ip 地址

分析一个 arp 的数据包

Ethernet II, Src: IntelCor_27:54:e3 (94:65:9c:27:54:e3), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
 Destination: Broadcast (ff:ff:ff:ff:ff:ff) 为获得某个IP地址的MAC地址，先进行广播
 Source: IntelCor_27:54:e3 (94:65:9c:27:54:e3)
 Type: ARP (0x0806) arp的标识

Address Resolution Protocol (request)
 Hardware type: Ethernet (1) 表示通过以太网指派
 Protocol type: IPv4 (0x0800) ipv4的标识
 Hardware size: 6
 Protocol size: 4
 Opcode: request (1)
 Sender MAC address: IntelCor_27:54:e3 (94:65:9c:27:54:e3)
 Sender IP address: 192.168.1.101
 Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00) 广播时全0，未填充 因为此时还不知道目的MAC地址
 Target IP address: 192.168.1.1

Ethernet II, Src: Shenzhen_0c:8d:62 (8c:f2:28:0c:8d:62), Dst: IntelCor_27:54:e3 (94:65:9c:27:54:e3)
 Destination: IntelCor_27:54:e3 (94:65:9c:27:54:e3) ARP请求中的源地址变为ARP应答中的目的地址
 Source: Shenzhen_0c:8d:62 (8c:f2:28:0c:8d:62)
 Type: ARP (0x0806)

Address Resolution Protocol (reply)
 Hardware type: Ethernet (1)
 Protocol type: IPv4 (0x0800)
 Hardware size: 6
 Protocol size: 4
 Opcode: reply (2) 此时为2，表示这是一个arp应答报文
 Sender MAC address: Shenzhen_0c:8d:62 (8c:f2:28:0c:8d:62) 并且此时应答附上了自己的mac地址
 Sender IP address: 192.168.1.1
 Target MAC address: IntelCor_27:54:e3 (94:65:9c:27:54:e3)
 Target IP address: 192.168.1.101

应发时这两个字段相同

3、ICMP 协议分析

icmp 报文可以通过 ping 命令来抓取，使用 wireshark 过滤 icmp 报文

No.	Time	Source	Destination	Protocol	Length	Info
8.846565	10.128.2.12	69.171.232.21	ICMP	74	Echo (ping) request	id=0x0001, seq=896/32771, ttl=128 (no response found!)
13.478059	10.128.2.12	69.171.232.21	ICMP	74	Echo (ping) request	id=0x0001, seq=897/33027, ttl=128 (no response found!)
18.478236	10.128.2.12	69.171.232.21	ICMP	74	Echo (ping) request	id=0x0001, seq=898/33283, ttl=128 (no response found!)
51.218580	10.128.2.12	112.80.248.75	ICMP	74	Echo (ping) request	id=0x0001, seq=899/33539, ttl=128 (reply in 6090)
51.233422	112.80.248.75	10.128.2.12	ICMP	74	Echo (ping) reply	id=0x0001, seq=899/33539, ttl=52 (request in 6089)
52.223284	10.128.2.12	112.80.248.75	ICMP	74	Echo (ping) request	id=0x0001, seq=900/33795, ttl=128 (reply in 6102)
52.238514	112.80.248.75	10.128.2.12	ICMP	74	Echo (ping) reply	id=0x0001, seq=900/33795, ttl=52 (request in 6101)
53.229216	10.128.2.12	112.80.248.75	ICMP	74	Echo (ping) request	id=0x0001, seq=901/34051, ttl=128 (reply in 6695)
53.245208	112.80.248.75	10.128.2.12	ICMP	74	Echo (ping) reply	id=0x0001, seq=901/34051, ttl=52 (request in 6610)
54.232470	10.128.2.12	112.80.248.75	ICMP	74	Echo (ping) request	id=0x0001, seq=902/34307, ttl=128 (reply in 6860)
54.248020	112.80.248.75	10.128.2.12	ICMP	74	Echo (ping) reply	id=0x0001, seq=902/34307, ttl=52 (request in 6859)

```
Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0x49d7 [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence number (BE): 900 (0x0384)
  Sequence number (LE): 33795 (0x8403)
  [Response frame: 6102]
  Data (32 bytes)
```

icmp echo request 报文结构

类型	代码	校验和
标识符		序列号
数据		

icmp 通用格式

类型	代码	校验和
不同的类型和代码,内容不同		

分析一个 icmp 报文

Internet Control Message Protocol

Type: 8 (Echo (ping) request)

Code: 0

Checksum: 0x49d5 [correct]

[Checksum Status: Good]

Identifier (BE): 1 (0x0001)

Identifier (LE): 256 (0x0100)

Sequence number (BE): 902 (0x0386)

Sequence number (LE): 34307 (0x8603)

[Response frame: 6860]

▼ Data (32 bytes)

Data: 6162636465666768696a6b6c6d6e6f707172737475767761...

[Length: 32]

表示icmp类型为查询报文

代码为0

校验和正确

标识符, BE为Linux主机, LE为Windows主机

序列号, BE为Linux主机, LE为windows主机

数据

Internet Control Message Protocol

Type: 0 (Echo (ping) reply)

Code: 0

Checksum: 0x51d5 [correct]

[Checksum Status: Good]

Identifier (BE): 1 (0x0001)

Identifier (LE): 256 (0x0100)

Sequence number (BE): 902 (0x0386)

Sequence number (LE): 34307 (0x8603)

[Request frame: 6859]

[Response time: 15.550 ms]

▼ Data (32 bytes)

Data: 6162636465666768696a6b6c6d6e6f707172737475767761...

[Length: 32]

响应报文

和请求为一组报文, 是一样的

差错报告 icmp 报文

